

Task-1.Design an event-driven workflow for a music-sharing app where users upload audio files and the app automatically extracts and stores the song's metadata (title, artist, duration) in a database.

User uploads audio



Amazon S3



S3 ObjectCreated event



AWS Lambda



Extract metadata

(title, artist, duration)



DynamoDB



Music metadata stored

How it works

1. The user uploads an MP3/audio file to an S3 bucket such as music-uploads-demo.
2. S3 generates an **ObjectCreated** event.
3. That event invokes a Lambda function such as ExtractMusicMetadata.
4. Lambda reads the audio file from S3 and extracts:
 - Title
 - Artist
 - Duration

5. Lambda stores the metadata in a DynamoDB table such as MusicMetadata.

Task-2. Implement a serverless function (such as AWS Lambda or Azure Function) that triggers when a new image is uploaded to a cloud storage bucket and writes the image's filename, upload time, and file size to a NoSQL database table.

Amazon S3 + AWS Lambda + DynamoDB

Step 1 — Create the DynamoDB table

Go to:

AWS Console → DynamoDB → Tables → Create table

Use:

Table name: ImageUploads

Partition key: fileName

Type: String

Click **Create table**.

Step 2 — Create the Lambda function

Go to:

Lambda → Create function

Choose:

Author from scratch

Use:

Function name: LogImageUpload

Runtime: Python 3.x

Create the function.

Step 3 — Give Lambda DynamoDB permissions

Go to:

Lambda → LogImageUpload → Configuration → Permissions

Open the Lambda execution role in IAM.

For a classroom/demo environment, attach:

AmazonDynamoDBFullAccess

For production, use a least-privilege policy allowing only the required PutItem operation on ImageUploads.

Step 4 — Lambda code

Replace the default Lambda code with:

```
import json
import boto3
import urllib.parse
from datetime import datetime, timezone

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table("ImageUploads")

def lambda_handler(event, context):

    print("Received S3 event:")
    print(json.dumps(event))

    for record in event.get("Records", []):

        bucket = record["s3"]["bucket"]["name"]
```

```
file_name = urllib.parse.unquote_plus(
    record["s3"]["object"]["key"]
)

file_size = record["s3"]["object"]["size"]

upload_time = datetime.now(timezone.utc).isoformat()

print(f"Bucket: {bucket}")
print(f"Filename: {file_name}")
print(f"File size: {file_size} bytes")
print(f"Upload time: {upload_time}")

table.put_item(
    Item={
        "fileName": file_name,
        "fileSize": file_size,
        "uploadTime": upload_time
    }
)

print("Image information saved to DynamoDB")

return {
    "statusCode": 200,
```

```
"body": json.dumps("Image upload processed successfully")
}
```

Click **Deploy**.

Step 5 — Create the S3 bucket

Go to:

S3 → Create bucket

For example:

image-upload-demo-yourname

Remember that S3 bucket names must be globally unique.

Upload an image such as:

profile.jpg

Step 6 — Add the S3 trigger

Open:

Lambda → LogImageUpload → Add trigger

Select:

Source: S3

Bucket: image-upload-demo-yourname

Event type: All object create events

You can optionally add a suffix filter:

.jpg

if you want Lambda to respond only to JPEG images.

Click **Add**.

Your workflow is now:

profile.jpg



S3



ObjectCreated



Lambda



DynamoDB

Step 7 — Test it

Upload another image to the bucket:

holiday.jpg

Wait a few seconds.

Then go to:

DynamoDB → ImageUploads → Explore table items

You should see something similar to:

fileName	fileSize	uploadTime
pinterest.jpg	245678	2026-10-06T...

You can also check:

Lambda → Monitor → View CloudWatch logs

aws

s3

Ask Amazon Q

Asia Pacific (Mumbai)

Mehta Darshit (8

Lambda > Functions > LogimageUploads

Successfully updated the function "LogimageUploads".

Function overview Info

Export to Infrastructure Composer Download

Diagram Template

LogimageUploads

Layers (0)

S3

Add trigger

Add destination

Function ARN

arn:aws:lambda:ap-south-1:856727778179:function:LogimageUploads

Description

-

Last modified

1 second ago

Learn how to implement use cases in AWS Lambda

Create a simple web app

In this tutorial you will how to:

- Build a simple web consisting of a Lam function with a fun URL that outputs a
- Invoke your functio through its function

Learn more

Start tutorial

Code Test Monitor Configuration Aliases Versions

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Mehta Darshit (856727778179)

Mehta Darshit

CloudWatch > Log management > /aws/lambda/LogimageUploads > 2026/10/06/[\$LATEST]0857a487e845466daafad5459f80e...

CloudWatch

Favorites and recents

Ingestion

Dashboards

Alarms

AI Operations

GenAI Observability

Application Signals (APM)

Infrastructure Monitoring

Logs

Log Management New

Log events

Actions Start tailing Create metric filter

You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns

Filter events - press enter to search

1m 1h UTC timezone

Display

Timestamp	Message
2026-10-06T10:46:31.697Z	Filename: Pinterest.jpg
2026-10-06T10:46:31.697Z	File size: 130288 bytes
2026-10-06T10:46:31.697Z	Upload time: 2026-10-06T10:46:31.697218+00:00
2026-10-06T10:46:32.268Z	[ERROR] ClientError: An error occurred (ValidationException) when calling the PutItem operat...
2026-10-06T10:46:32.288Z	END RequestId: 6421b9e7-956f-4121-a288-0ad8e5cf7f53
2026-10-06T10:46:32.288Z	REPORT RequestId: 6421b9e7-956f-4121-a288-0ad8e5cf7f53 Duration: 591.61 ms Billed Duration: ...

No newer events at this moment. Auto retry paused. Resume

Back to top

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

33°C Sunny

Search

4:14 PM 06-Oct-26

image-upload-demo-dm Info

Objects (2)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	download (1).jpg	jpg	October 6, 2026, 16:09:05 (UTC+05:30)	159.7 KB	Standard
☐	Pinterest.jpg	jpg	October 6, 2026, 16:16:31 (UTC+05:30)	127.2 KB	Standard

DynamoDB Tables

Tables (1) Info

Find tables

Filter by tag: Any tag key

Filter by tag value: Any tag value

☐	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection
☐	ImageUploads	Active	Filename (S)	-	0	0	Off

Task-3. Build a notification system for a Zomato-style food delivery app: when a new menu PDF is uploaded by a restaurant, trigger a function that sends a confirmation email to the restaurant owner and logs the event in a database. Use cloud storage event triggers and a transactional email API (like SendGrid or AWS SES) for the email step.

Step 1 — Create the S3 bucket

Create something like:

restaurant-menu-uploads-yourname

Restaurant owners upload their PDF menus here.

For example:

restaurant123-menu.pdf

Step 2 — Create a DynamoDB table

Create:

MenuUploadEvents

Use:

Partition key: eventId

Type: String

The table will store records such as:

```
{  
  "eventId": "restaurant123-menu.pdf-20261006",  
  "fileName": "restaurant123-menu.pdf",  
  "restaurantId": "restaurant123",  
  "uploadedAt": "2026-10-06T10:45:00Z",  
  "status": "EMAIL_SENT"  
}
```

Step 3 — Configure Amazon SES

Open:

Amazon SES

Before using SES, you'll need to configure/verify the sender identity according to your AWS account's SES requirements.

For a lab, you can verify an email address that you control.

For example:

restaurant-notifications@example.com

You should also verify the recipient address if your SES account is operating in the SES sandbox.

Step 4 — Create the Lambda function

Create:

MenuUploadNotification

Runtime:

Python 3.x

The Lambda needs permissions for:

S3 read

DynamoDB PutItem

SES SendEmail

CloudWatch Logs

Step 5 — Lambda code

Here's a simplified implementation:

```
import json
```

```
import boto3
```

```
import urllib.parse
```

```
import uuid
```

```
from datetime import datetime, timezone
```

```
dynamodb = boto3.resource("dynamodb")
```

```
table = dynamodb.Table("MenuUploadEvents")
```

```
ses = boto3.client("ses")
```

```
RESTAURANT_OWNER_EMAIL = "your-verified-email@example.com"
```

```
SENDER_EMAIL = "your-verified-email@example.com"
```

```
def lambda_handler(event, context):
```

```
    print("Received menu upload event")
```

```
    print(json.dumps(event))
```

```
    for record in event.get("Records", []):
```

```
        bucket = record["s3"]["bucket"]["name"]
```

```
        file_name = urllib.parse.unquote_plus(
            record["s3"]["object"]["key"]
        )
```

```
        upload_time = datetime.now(timezone.utc).isoformat()
```

```
        event_id = str(uuid.uuid4())
```

```
        print(f"Menu uploaded: {file_name}")
```

```
# Log event in DynamoDB
```

```
table.put_item(
```

```
    Item={
```

```
        "eventId": event_id,
```

```
        "fileName": file_name,
```

```
        "bucket": bucket,
```

```
        "uploadedAt": upload_time,
```

```
        "status": "UPLOADED"
```

```
    }
```

```
)
```

```
# Send confirmation email
```

```
ses.send_email(
```

```
    Source=SENDER_EMAIL,
```

```
    Destination={
```

```
        "ToAddresses": [
```

```
            RESTAURANT_OWNER_EMAIL
```

```
        ]
```

```
    },
```

```
    Message={
```

```
        "Subject": {
```

```
            "Data": "Menu uploaded successfully"
```

```
        },
```

```
        "Body": {
```

```
            "Text": {
```

```
                "Data": (
```

```
        f"Your menu file '{file_name}' "  
        "was successfully uploaded."  
    )  
}  
}  
}  
)
```

```
# Update status
```

```
table.update_item(  
    Key={  
        "eventId": event_id  
    },  
    UpdateExpression="SET #s = :status",  
    ExpressionAttributeNames={  
        "#s": "status"  
    },  
    ExpressionAttributeValues={  
        ":status": "EMAIL_SENT"  
    }  
)
```

```
print("Confirmation email sent")
```

```
return {  
    "statusCode": 200,
```

```
"body": json.dumps("Menu upload processed")
}
```

Replace:

your-verified-email@example.com

with an email address you've verified with SES.

Step 6 — Configure the S3 trigger

In Lambda:

MenuUploadNotification → Add trigger → S3

Select:

Bucket:

restaurant-menu-uploads-yourname

Event:

All object create events

For this assignment, you can also configure a suffix filter:

.pdf

This means the Lambda won't run for unrelated files such as JPG images.

Step 7 — Test

Upload:

restaurant123-menu.pdf

S3 triggers Lambda.

Lambda:

1. Reads the event.
2. Gets the filename.
3. Records the upload in DynamoDB.

4. Sends the restaurant owner a confirmation email.
5. Updates the DynamoDB record to EMAIL_SENT.

CloudWatch should contain:

Menu uploaded: restaurant123-menu.pdf

Confirmation email sent

Production improvement

For a real food-delivery application, I wouldn't make the S3-triggered Lambda directly perform every operation synchronously. A more resilient design would be:

S3



Lambda



SQS



Notification Lambda



SES DynamoDB

An SQS queue provides buffering and allows failed notification attempts to be retried without losing the event.

Task-4. Review a classmate's event-driven workflow diagram and provide 2 suggestions for improving scalability or error handling in their design.

Add an SQS queue or dead-letter queue (DLQ) between the event producer and processing Lambda. If Lambda or DynamoDB fails temporarily, the event can be retried instead of being lost. A DLQ can hold messages that continue to fail so they can be investigated later.

Example:

S3



SQS



Lambda



DynamoDB